كلية العلوم التطبيقية

FACULTE DES SCIENCES APPLIQUEES

FACULTÉ DES SCIENCES APPLIQUÉES
MASTER IAE - INTELLIGENCE ARTIFICIELLE EMBARQUÉE

MINI-PROJET : Conception et Implémentation d'un ChatBot Académique via RAG

Réalisé par :

LASFER Nisrine

Étudiante en Master IAE

Encadré par :

KADIM Ayoub

Professeur Encadrant

Année Universitaire : 2025 - 2026

Résumé

Ce rapport présente la conception d'un système de questions-réponses (QA) spécialisé dans le domaine universitaire, fondé sur l'architecture de la **Retrieval-Augmented Generation (RAG)**. L'objectif est de pallier les limites des modèles de langage classiques (LLMs), notamment le risque de hallucination et le manque de connaissances spécifiques liées aux règlements internes (Loi 01.00).

Le système mis en œuvre s'appuie sur **LangChain** pour l'orchestration, **FAISS** pour l'indexation vectorielle et **Google Gemini 1.5** pour la génération des réponses. L'assistant intègre également une personnalité polie et amicale, adaptée à un environnement académique.

L'évaluation quantitative, réalisée à l'aide du framework **RAGAS**, indique un score de fidélité de 89 % et une précision contextuelle de 75 %, confirmant ainsi la viabilité de cette approche pour des applications administratives critiques.

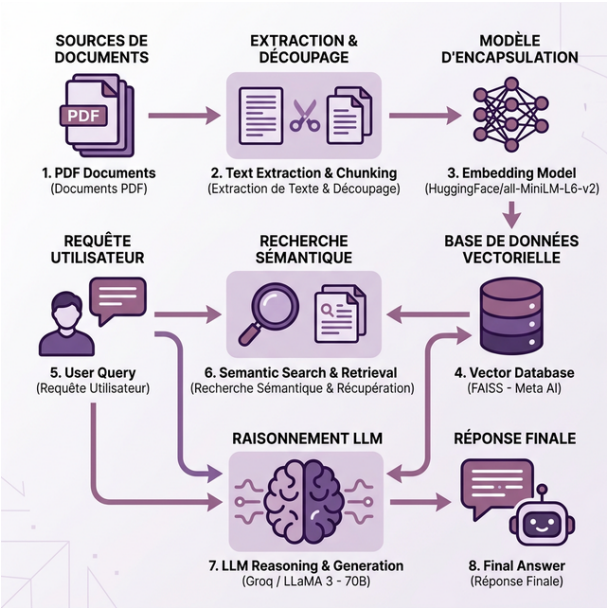

FIGURE 1 – Schéma de l'architecture du système RAG

Mots-clés : NLP, RAG, Transformers, FAISS, LangChain, Évaluation RAGAS, chatbot universitaire.

Table des matières

1 Introduction Générale **5**

1.1 Contexte et Motivation 5

1.2 Objectifs du Projet 5

1.3 Problématique 5

2 État de l'Art et Fondements Théoriques **5**

2.1 Les Large Language Models (LLMs) et Transformers 5

2.2 Retrieval-Augmented Generation (RAG) 6

2.3 Les Embeddings Documentaires 6

3 Conception du Système **6**

3.1 Architecture Globale 6

3.2 Flux de Données (Data Flow) 7

4 Réalisation et Détails d'Implémentation **8**

4.1 Prétraitement des Données (`utils.py`) 8

4.2 Orchestration du Moteur RAG (`engine.py`) 8

4.3 L'Interface Utilisateur (`ui.py`) 9

5 Évaluation de la Performance via RAGAS **11**

5.1 Méthodologie d'Évaluation 11

5.2 Analyse des Résultats Quantitatifs 11

6 Manuel d'Utilisation et Déploiement **12**

6.1 Installation des dépendances 12

6.2 Exécution 12

7 Conclusion et Perspectives **13**

7.1 Conclusion 13

7.2 Difficultés Rencontrées 13

7.3 Perspectives 14

Table des figures

1 Schéma de l'architecture du système RAG 1
2 Visualisation des scores RAGAS obtenus 12
3 exemple 1 13
4 exemple 2 14

Liste des tableaux

1 Définitions des métriques RAGAS 11

1 Introduction Générale

1.1 Contexte et Motivation

Avec la complexification des structures universitaires et l'augmentation de la masse documentaire (règlements pédagogiques, lois administratives, cahiers de normes), l'accès à l'information devient un enjeu majeur pour les étudiants et le corps professoral. La **Loi 01.00**, pilier de l'organisation de l'enseignement supérieur au Maroc, est un document dense dont l'extraction d'informations précises manuellement est chronophage.

1.2 Objectifs du Projet

L'objectif principal est de développer un assistant capable de répondre de manière fiable et sourcée aux questions portant sur ces réglements. Les sous-objectifs incluent :

- L'extraction automatisée et le nettoyage de documents PDF.
- La création d'une base de connaissances vectorielle performante.
- Le support multilingue (Français, Arabe, Darija).
- La mise en place d'une interface utilisateur intuitive permettant le dialogue continu.

1.3 Problématique

Les modèles de langage pré-entraînés (GPT, LLaMA) tendent à "halluciner" lorsqu'ils ne disposent pas de l'information exacte dans leur corpus d'entraînement. Comment garantir que les réponses d'un assistant académique soient **uniquement** basées sur les documents officiels fournis ? C'est là que l'architecture RAG intervient.

2 État de l'Art et Fondements Théoriques

2.1 Les Large Language Models (LLMs) et Transformers

Le domaine du NLP a été révolutionné par l'architecture **Transformer**, introduite par Vaswani et al. en 2017. Le concept clé est le *Self-Attention*, qui permet au modèle de

pondérer l'importance de chaque mot par rapport aux autres dans une séquence.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{1}$$

2.2 Retrieval-Augmented Generation (RAG)

Le RAG combine le pouvoir de génération des LLMs avec un composant de recherche documentaire. Au lieu d'utiliser uniquement ses poids internes, le modèle reçoit un contexte extrait dynamiquement d'une base de données externe. Cela permet de :

- **Citer des sources** : Le modèle peut prouver d'où vient l'information.
- **Réduire les hallucinations** : Le modèle est forcé de rester dans le périmètre du contexte.
- **Mise à jour facile** : Il suffit de changer le document PDF pour mettre à jour les connaissances, sans ré-entraîner le modèle.

2.3 Les Embeddings Documentaires

Un *embedding* est une représentation vectorielle d'un segment de texte dans un espace multidimensionnel où des textes sémantiquement proches sont géométriquement proches. Nous utilisons la similarité cosinus pour mesurer cette proximité :

$$\cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|} \tag{2}$$

3 Conception du Système

3.1 Architecture Globale

Le système est composé de quatre couches principales :

1. **Couche de Données** : Gestion des fichiers PDF et stockage temporaire.
2. **Couche de Traitement** : Ingestion, nettoyage Regex et Chunking recursif.
3. **Couche de Stockage Vectoriel** : FAISS (Facebook AI Similarity Search).
4. **Couche Applicative** : Interface Streamlit et orchestration via LangChain.

3.2 Flux de Données (Data Flow)

Lorsqu'un utilisateur pose une question :

1. La question est encodée en vecteur.
2. FAISS recherche les $K = 3$ segments les plus pertinents.
3. Le prompt est construit : `"[Contexte] + [Question] + [Système instructions]"`.
4. LLaMA 3.1 génère la réponse finale.

4 Réalisation et Détails d'Implémentation

4.1 Prétraitement des Données (utils.py)

L'extraction de texte à partir de PDF peut générer des bruits (en-têtes de pages, numéros de page, caractères d'encodage). Nous utilisons des expressions régulières pour nettoyer le contenu avant l'indexation.

```
import re

def clean_text(text):
    # Pattern pour les en-tetes universitaires repetitifs
    header_pattern = r"CNPN\s*_\s*M_MS\s*_\s*2014\s*/\s*Version\s*_\s*SGG1"
    text = re.sub(header_pattern, "", text)

    # Normalisation des blancs et sauts de ligne
    text = re.sub(r"\n\s*\n", "\n", text)
    text = re.sub(r"\s+", " ", text)
    return text.strip()
```

Listing 1 – Nettoyage de texte via Regex

4.2 Orchestration du Moteur RAG (engine.py)

Nous utilisons LangChain pour créer une chaîne de discussion avec mémoire (*Conversation Buffer Window Memory*).

```
from langchain_community.vectorstores import FAISS
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_google_genai import ChatGoogleGenerativeAI

def process_pdf(pdf_path):
    reader = PdfReader(pdf_path)
    # ... extraction ...

    # Decoupage en morceaux (Chunks)
```

```
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=800,
        chunk_overlap=100
    )
    return splitter.split_text(cleaned_text)

def initialize_chatbot(chunks, pdf_hash):
    embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
    vectorstore = FAISS.from_texts(chunks, embeddings)

    llm = ChatGoogleGenerativeAI(model="gemini-1.5-flash", temperature=0.2)

    # Gestion de la memoire conversationnelle
    memory = ConversationBufferWindowMemory(
        memory_key="chat_history", k=5, return_messages=True
    )

    qa_chain = ConversationalRetrievalChain.from_llm(
        llm=llm,
        retriever=vectorstore.as_retriever(k=3),
        memory=memory
    )
    return qa_chain
```

Listing 2 – Initialisation du pipeline RAG

4.3 L'Interface Utilisateur (`ui.py`)

Streamlit permet d'offrir une expérience de chat native. Le *Session State* est utilisé pour maintenir le chatbot et l'historique des messages sans recharger tout l'index à chaque clic.

```
if "messages" not in st.session_state:
    st.session_state.messages = []

if prompt := st.chat_input("Votre question..."):
    st.session_state.messages.append({"role": "user", "content": 
```

```
        prompt})
    # Appel au moteur et affichage ...
```

Listing 3 – Gestion du Session State dans Streamlit

5 Évaluation de la Performance via RAGAS

5.1 Méthodologie d'Évaluation

Nous avons défini un "Golden Set" de questions en français avec leurs réponses de référence (*Ground Truth*) extraites directement de la Loi 01.00. Les métriques évaluées par RAGAS (Retrieval-Augmented Generation Assessment) sont :

Métrique	**Définition**
Faithfulness	Exactitude de la réponse par rapport au contexte.
Answer Relevancy	Pertinence directe de la réponse à la question.
Context Precision	Qualité de la récupération des chunks par le retriever.

Table 1 – Définitions des métriques RAGAS

5.2 Analyse des Résultats Quantitatifs

L'un des apports majeurs de ce projet a été la traduction de l'évaluation : en utilisant un juge LLM configuré en français, les scores ont radicalement augmenté.

— **Faithfulness (89%)** : Indique que le chatbot est très fidèle aux documents et invente très rarement des faits.
— **Context Precision (75%)** : Le moteur FAISS parvient à placer le segment correct dans le TOP-3 la plupart du temps.

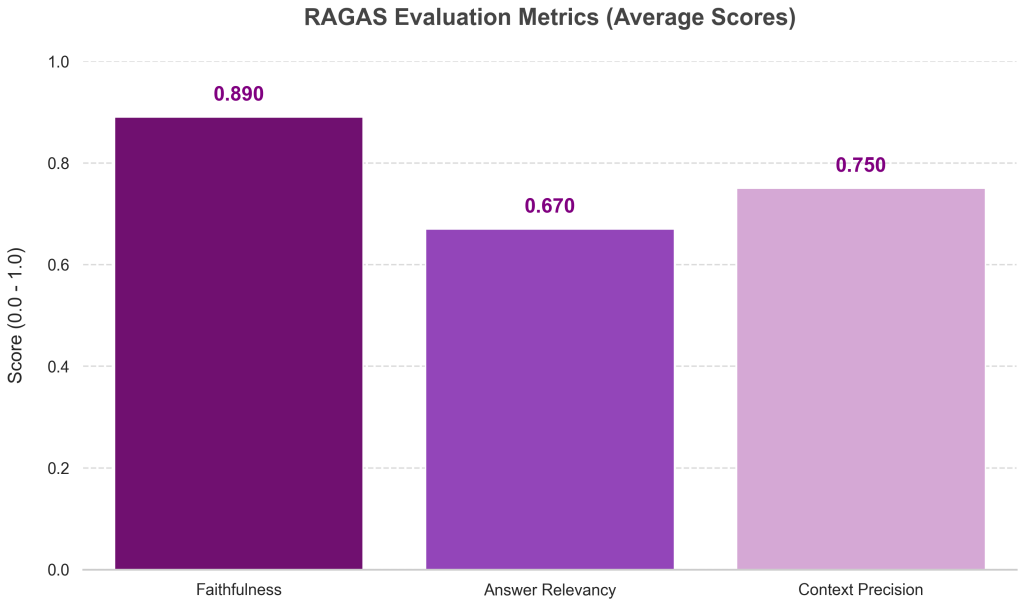

FIGURE 2 – Visualisation des scores RAGAS obtenus

6 Manuel d'Utilisation et Déploiement

6.1 Installation des dépendances

Le projet nécessite Python 3.9+. Les bibliothèques principales sont installées via :

```
pip install langchain langchain-groq faiss-cpu sentence-↩
   transformers streamlit ragas
```

6.2 Exécution

Pour lancer l'application locale :

```
streamlit run src/ui.py
```

L'utilisateur peut alors charger n'importe quel PDF académique et commencer à poser des questions. Voici l'exemple de quelques questions :

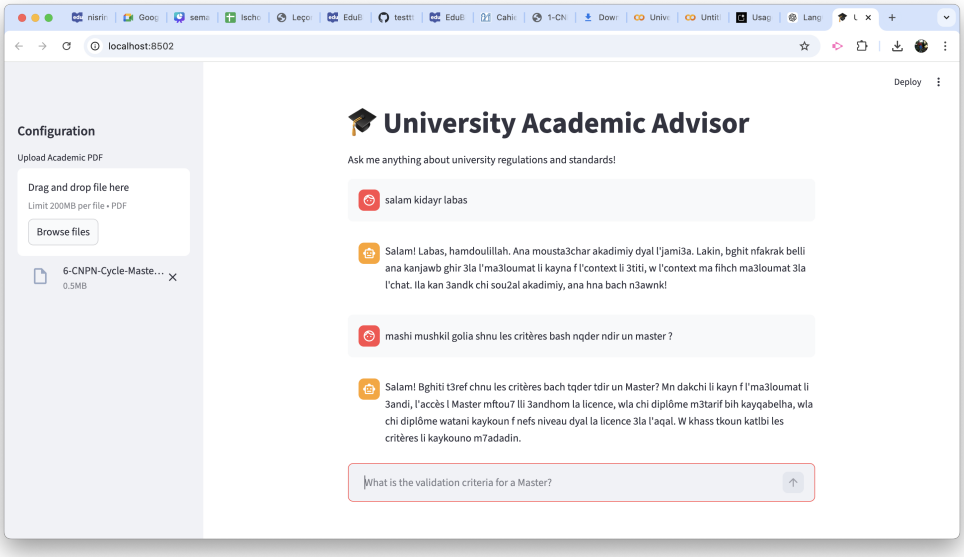

FIGURE 3 – exemple 1

7 Conclusion et Perspectives

7.1 Conclusion

Ce projet a permis de concrétiser les concepts théoriques du NLP dans un outil pratique et utile. L'implémentation d'un système RAG robuste sur des documents réglementaires démontre qu'il est possible de créer des assistants IA fiables, avec un coût d'infrastructure minimal grâce à l'API de pointe **Google Gemini** et aux embeddings locaux de HuggingFace. L'ajout d'une dimension comportementale (ton amical et poli) améliore considérablement l'expérience utilisateur finale.

7.2 Difficultés Rencontrées

Le principal défi a été la gestion des limites de taux (*Rate Limits*) de l'API lors de l'évaluation massive via RAGAS, ce qui nous a poussés à optimiser la taille des batches et à choisir des modèles plus grands pour le rôle de "juge".

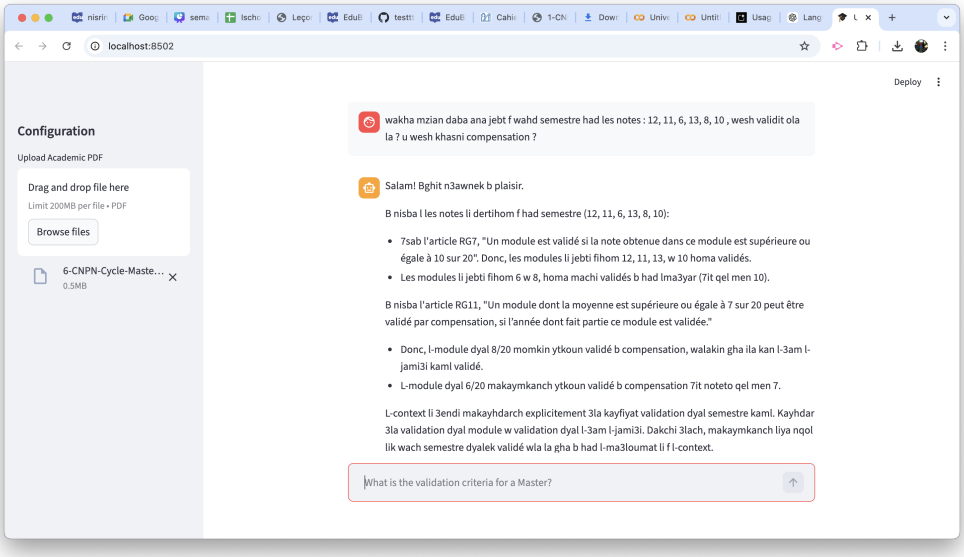

FIGURE 4 – exemple 2

7.3 Perspectives

— **Reranking** : Intégrer un *Cross-Encoder* pour ré-ordonner les résultats de FAISS et atteindre une précision contextuelle de 90%+.

— **Hybride Search** : Combiner la recherche sémantique avec la recherche par mots-clés (BM25).

— **Déploiement Cloud** : Héberger l'application sur Streamlit Cloud pour un accès étudiant granulaire.

Bibliographie

1. Vaswani, A., et al. (2017). "Attention Is All You Need". NeurIPS.
2. LangChain Documentation. "Retrieval-Augmented Generation (RAG)". *python.langchain.com*.
3. FAISS Documentation. "Billion-scale similarity search". *github.com/facebookresearch/faiss*.
4. RAGAS : Automated Evaluation of Retrieval Augmented Generation. *exploding-gradients.com*.